

Twenty-Sixth Implementation Iteration: Request-Type Permission Rule with Separate Authorization for Request Submission

Software Design / Iteration Report

Name: Najib Attar

Date: May 2026

1. Iteration Goal

The goal of this twenty-sixth implementation iteration is to add request-type permission checking to the resource allocation system.

Before this iteration, `ExamRequest` existed as a first-class request subtype, but the system did not explicitly control which user types were allowed to submit exam requests. This meant that a student or staff member could submit an exam request and only later be rejected by other rules, such as space authorization or capacity.

In this iteration, a new rule called `RequestTypeRule` is introduced. This rule checks whether the requester is allowed to submit the specific request type. It is separate from `UserRoleRule`, which continues to check whether the user is allowed to use the requested space type.

2. Changes from Iteration 25

Compared with the twenty-fifth iteration, the following changes were introduced:

- Added `canSubmitRequestType(...)` to the polymorphic `User` base class
- Implemented request-type permission behavior in:
 - `Student`
 - `Instructor`
 - `Staff`
 - `Administrator`
- Added `getRequestType()` to the `Request` hierarchy
- Implemented request type strings:

- `OneTimeRequest` returns `OneTime`
- `RecurringRequest` returns `Recurring`
- `ExamRequest` returns `Exam`
- `InvalidRequest` returns `Invalid`
- Added a new rule class: `RequestTypeRule`
- Registered `RequestTypeRule` early in the rule evaluation pipeline
- Updated `requests.csv` with exam request examples from different user types

This iteration does not change request creation, allocation strategy behavior, CSV export behavior, or exam allocation logic.

3. Scenario

The prototype begins by loading users, spaces, requests, and configuration data through the existing data loading workflow. Requests are still created through `RequestFactory`, and exam requests are still represented using `ExamRequest`.

During request processing, the selected allocation strategy sends each valid request to the rule engine through `RuleEngineFacade`. The new `RequestTypeRule` runs early in the rule list. It asks the requester whether the current request type is allowed by calling `canSubmitRequestType(...)`.

If the requester is not allowed to submit that request type, the request is rejected immediately with a clear rejection reason. For example, a student submitting an `ExamRequest` is rejected with a message explaining that the user is not allowed to submit exam requests. If the request type is allowed, the system continues with the existing checks, including space-type authorization, capacity, feature, status, location, and availability.

4. Modules and Their Tasks

A. User Permission Module

- Store request-type submission behavior inside user subclasses
- Allow each user type to decide which request types it may submit
- Keep request-type permissions polymorphic and extensible

B. Request Type Identification Module

- Add a consistent request type string to the request hierarchy
- Separate request type from request purpose
- Allow rules to evaluate request type without relying on metadata fields

C. RequestTypeRule Module

- Check whether a requester may submit the current request type
- Reject unauthorized request types with a meaningful reason
- Remain separate from `UserRoleRule`

D. Rule Engine Facade Module

- Register `RequestTypeRule` as part of the common rule evaluation flow
- Run request-type permission before later validation checks
- Preserve the existing rule engine structure

E. Data and Reporting Module

- Preserve request loading through `RequestFactory`
- Preserve console output and CSV export behavior
- Export request-type rejection reasons through the existing history and result fields

5. Request-Type Permission Policy

The request-type permission policy implemented in this iteration is as follows:

- **Student:** can submit `OneTime` and `Recurring`; cannot submit `Exam`
- **Instructor:** can submit `OneTime`, `Recurring`, and `Exam`
- **Staff:** can submit `OneTime` and `Recurring`; cannot submit `Exam` for now
- **Administrator:** can submit all supported request types

This policy is separate from space-type authorization. For example, `RequestTypeRule` decides whether a user may submit an exam request, while `UserRoleRule` still decides whether that user may request a classroom, laboratory, or meeting room.

6. Iteration Comparison

Iter.	Focus	Added / Modified	Design bility	Responsi-	Prototype Effect
22	Polymorphic users	User hierarchy and UserFactory	Moved user-specific role, priority, and space authorization into user classes.		System behavior became user-driven and easier to extend for different user types.
23	Request creation	RequestFactory	Separated request construction from DataController .		The controller became cleaner and less responsible for object creation.
24	Request meta-data	title and purpose in Request	Extended requests with descriptive information for future use cases.		Requests became clearer and better prepared for exam-oriented extensions.
25	Explicit exam requests	ExamRequest and exam-related fields	Separated exam-specific meaning from generic one-time requests.		Exams became first-class requests while still using one-time allocation behavior.
26	Request-type permission	RequestTypeRule , canSubmitRequestType() , and getRequestType()	Separated request submission permission from space-type authorization.		Students and staff can now be rejected for submitting exam requests before other rule checks continue.

7. Updated Class Diagram

Figure 1 shows the updated class diagram. The main additions in this iteration are `RequestTypeRule`, `User::canSubmitRequestType(...)`, and `Request::getRequestType()`. The diagram also shows that `RequestTypeRule` is separate from `UserRoleRule`.

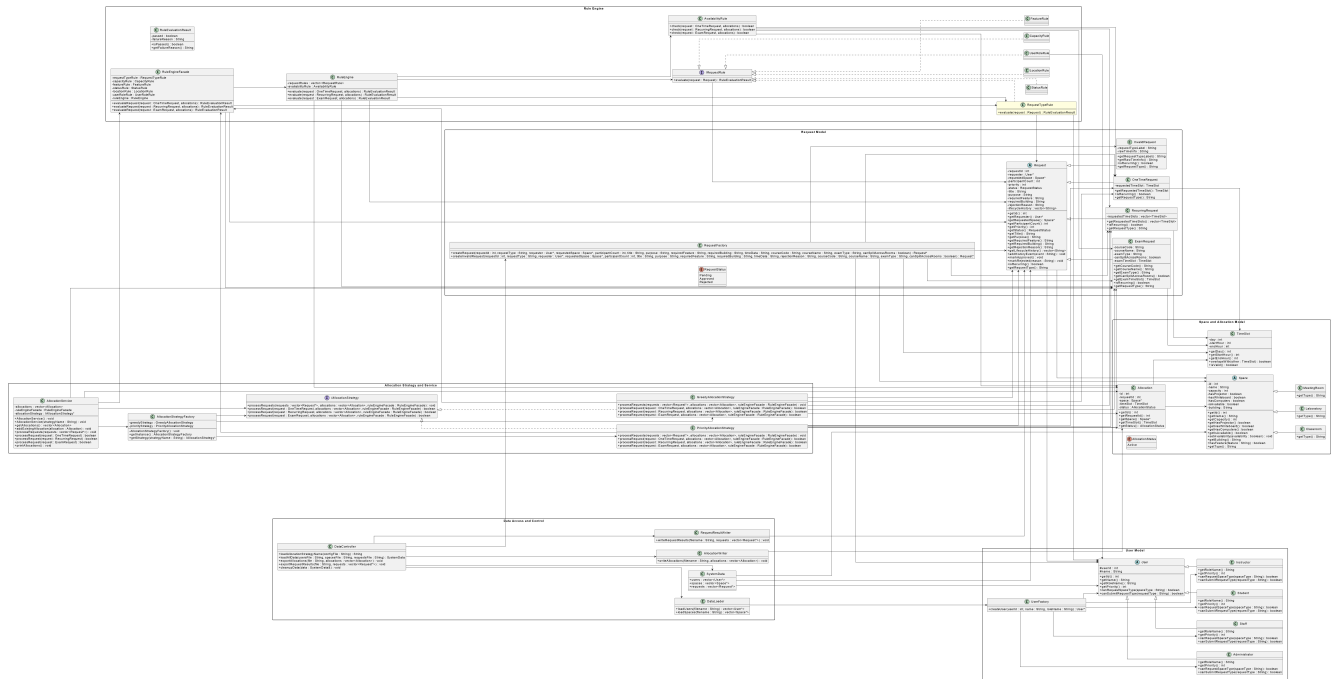


Figure 1: Updated Class Diagram with `RequestTypeRule`

8. Updated Workflow Diagram

Figure 2 shows the updated workflow. During rule evaluation, `RequestTypeRule` runs early. If the requester is not allowed to submit the request type, the request is rejected. If the request type is allowed, the normal space-type authorization and other validation rules continue.

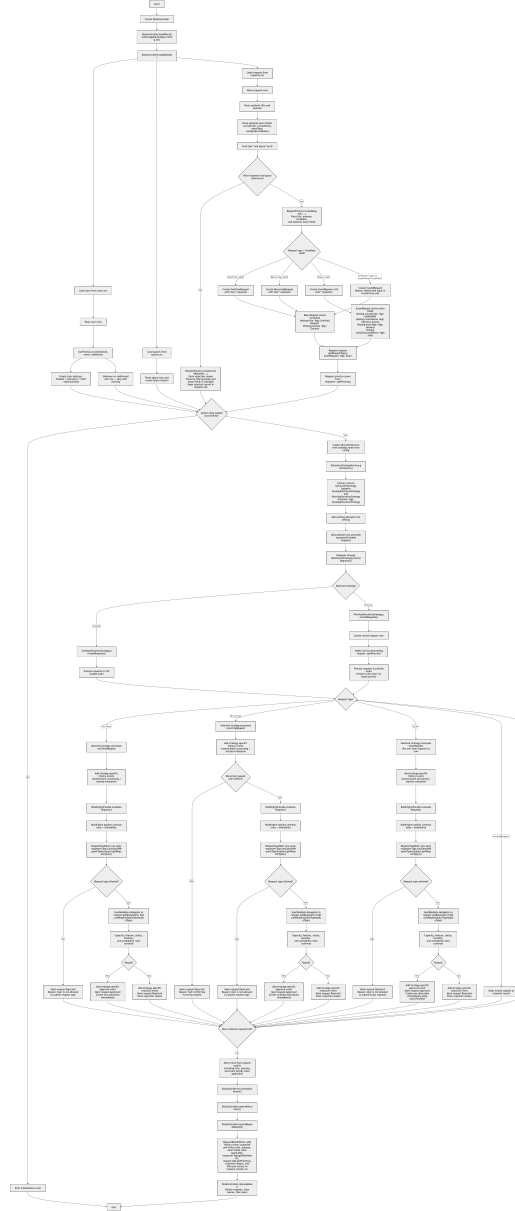


Figure 2: Workflow with Request-Type Permission Checking

9. Prototype Implementation

The C++ prototype was extended by adding a new polymorphic user behavior method, `canSubmitRequestType(...)`. Each concrete user subclass now defines which request

types it may submit. This keeps the permission policy close to the user type that owns the behavior.

The request hierarchy was also extended with `getRequestType()`. This method returns a model-level request type such as `OneTime`, `Recurring`, `Exam`, or `Invalid`. This is intentionally separate from `getPurpose()`, because a request purpose may describe why a space is needed, while the request type identifies the concrete request model.

A new `RequestTypeRule` class was added to the rule system. It implements the existing `IRequestRule` interface and evaluates a request by checking whether the requester can submit that request type. If not, it returns a failed `RuleEvaluationResult` with a clear rejection reason.

`RuleEngineFacade` was updated to include `RequestTypeRule` before `UserRoleRule`. This ordering allows the system to reject unauthorized exam submissions early, while still preserving the existing space-type authorization rule. The old enum-based role system remains removed; `UserRoleRule` is only the name of the rule class and still delegates to polymorphic user behavior.

The sample request data was extended with exam requests from different user types. Instructor and administrator exam requests pass the request-type permission check, while student and staff exam requests are rejected by `RequestTypeRule`. The rejection reason is preserved in console output, `request_results.csv`, and request lifecycle history.

10. Summary

This twenty-sixth implementation iteration adds request-type authorization to the resource allocation system. The new `RequestTypeRule` checks whether a requester is allowed to submit a request type such as `Exam`, while `UserRoleRule` continues to check whether the requester is allowed to use the requested space type.

The change keeps responsibilities separated and prepares the system for future exam-specific workflows. Students and staff are now prevented from submitting exam requests, while instructors and administrators can continue submitting them. Existing request creation, allocation strategies, CSV loading and exporting, request history, and one-time exam processing behavior remain unchanged.